



- The name of our project:

“Logic-based Travel planner”

ID	Name	Group
100308	Hosam Khaled Bahnasy Dyab	G3-CS
101537	Hazem Mohamed Zaki Labib	G3-CS
102747	Khaled Gamal Saber Fouda	G3-CS
100614	Basmala Ayman Abdelhalim Mohamed	G2-CS
99690	Aya Mohamed Salah Eldin	G2-CS
101602	Anas Magdy Saied Elkhafif	G2-CS
102086	Abdulrahman Ghandour Mohammed	CASE

Supervised by:

Prof. Dr. Hesham ElDeeb
T.A. Nardin Wagih

Table of Contents

1. Problem Definition	4
2. Project Description.....	4
Overview of Prolog Code	4
Facts	5
1.hotel/5	5
2.hotel_review/3.....	6
3.transport/5.....	6
4.activity/6	7
5.attraction/4.....	8
6.daily_time_constraint/1	9
7.food_cost/2.....	9
8.month_to_number/2.....	10
9.season/2.....	10
10.city/3	11
11.activity_attraction_link/2	12
Rules.....	13
1.start_planner/0.....	13
2.validate_inputs/5.....	13
3.check_season/3	14
4.select_hotel/7	15
5.select_transport/3	16
6.select_food_cost/4	17
7.collect_all_activities/2	18
8.distribute_activities/8.....	18
9.distribute_activities_by_day/10	19
10.select_daily_activities/6	20
11.calculate_total_cost/5	21
12.check_budget/4	22
13.print_travel_plan/14.....	23
14.submit_hotel_review/4	24
15.update_hotel/5	25
The Output in SWI-Prolog:.....	27
The Output with using GUI:	28
1.Diagrams	29
3.1 Use Case Diagram	29
3.1.1 Actors	29

3.1.2 Primary Use Cases	30
3.1.3 Extended Use Cases	30
3.2 Sequence Diagram	31
3.2.1 Actors and Participants	33
3.2.2 Sequence Flow	33
3.2.3 Control Structures	34
3.3 Activity Diagram	34
3.3.1 Workflow Overview	35
3.3.2 Activity Flow	36
3.3.3 Control Structures	37
4. Conclusion	37
5. References	38
6. Our Project in GitHub	38

Figure 1: Hotel/5	5
Figure 2: Hotel_review/3	6
Figure 3: transport/5	7
Figure 4: activity/6	8
Figure 5: attraction/4	9
Figure 6: daily_time_constraint/1	9
Figure 7: food_cost/2	10
Figure 8: month_to_number/2	10
Figure 9: season/2	11
Figure 10: city/3	12
Figure 11: activity_attraction_link/2	12
Figure 12: start_planner	13
Figure 13: validate_inputs/5	14
Figure 14: check_season/3	15
Figure 15: select_hotel/7	16
Figure 16: select_transport/3	16
Figure 17: select_food_cost/4	17
Figure 18: collect_all_activities/2	18
Figure 19: collect_all_activities/2	19
Figure 20: distribute_activities_by_day/10	20
Figure 21: select_daily_activities/6	21
Figure 22: calculate_total_cost/5	22
Figure 23: check_budget/4	22
Figure 24: print_travel_plan/14	24
Figure 25: submit_hotel_review/4	25
Figure 26: update_hotel/5	26
Figure 27: Use case diagram	29
Figure 28: Sequence diagram	32
Figure 29: Activity diagram	35

1. Problem Definition

Before we had *logic-based systems* like **Prolog** to help with travel planning, it wasn't easy for travelers to get their trips organized efficiently. People often struggled to find *trustworthy recommendations* for **destinations**, had a tough time trying to **accurately budget**, and couldn't easily gather **info** about cities, like *landmarks*, *tourist spots*, and *cultural attractions*. When they finally arrived at their destination, they faced even more challenges, like picking **budget-friendly hotels**, figuring out **transportation options**, and deciding on fun **activities** such as *shopping*, catching a film at the *cinema*, or going on *museum tours*. All these obstacles made travel planning a *time-consuming*, *confusing*, and often *frustrating* process. The **Logic-based Travel Planner** steps in to tackle these problems by offering an *automated*, **logic-driven framework** that makes decision-making easier, optimizes **how resources are used**, and really enhances the travel experience.

2. Project Description

The **Logic-based Travel Planner** is a cool *software tool* that aims to change the way we plan our trips by giving you **customized**, *personalized travel itineraries*. Built as a key project for the *Logic Programming course*, this system makes use of **Prolog's declarative programming model** to make the planning process *smooth* and *well-organized*. It helps travelers by **calculating budgets** based on how long they'll be away (whether it's days or weeks), **recommending hotels** that fit their *financial limits*, and pointing out **top-rated places to stay** in the city they're visiting. Users can pick from **various cities or countries** and get **in-depth info** about *things to see*—from *iconic landmarks* to *tourist hotspots* and *cultural spots*—while also getting help with **transportation** and **custom suggestions** for *hotels*, *attractions*, and *activities* that match their preferences. And it's not just about logistics; the system makes your trip more enjoyable by helping you discover **fun activities**, like *shopping*, *movie outings*, and *museum visits*, so you can skip tedious *manual research*.

Overview of Prolog Code

The Logic-based Travel Planner is a Prolog-based *expert system* designed to generate **personalized** travel itineraries for users visiting Egyptian cities such as Cairo, Aswan, and Port Said. The system employs a robust *knowledge base* of facts and a set of rules to *automate* travel planning, optimizing for **budget**, *time*, and user preferences. This document provides a detailed description of the facts and rules as implemented in the provided Prolog code, aligning with the predicates outlined in the `TravelPlanner_description.txt` document. The descriptions cover the structure, functionality, and purpose of each fact and rule, ensuring a **comprehensive** understanding of the system's *logic*.

Facts

The *knowledge base* consists of structured Prolog facts that encapsulate **travel-related** data. These facts serve as the foundational data queried and manipulated by the system's rules to generate **travel plans**.

1. hotel/5

- **Structure:** `hotel(City, Name, PricePerNight, Rating, Demand)`
- **Description:** Represents hotels available in each city, capturing **essential** details for *accommodation* planning.
- **Arguments:**
 - City: The city where the hotel is located (e.g., 'Cairo', 'Aswan', 'PortSaid').
 - Name: The name of the hotel (e.g., 'Hilton Cairo Nile').
 - PricePerNight: The cost per night in EGP (e.g., 1200).
 - Rating: The hotel's quality rating on a 0–5 scale (e.g., 4.7).
 - Demand: The demand level ('high' or 'low'), indicating *popularity* or *availability*.
- **Example:** `hotel('Cairo', 'Hilton Cairo Nile', 1200, 4.7, high)`
- **Purpose:** Provides data for hotel selection, enabling the system to filter **accommodations** based on *budget*, *rating*, and *city*.

```
% --- Facts ---
% (Facts remain the same as before - omitted for brevity)
% hotel(City, Name, PricePerNight, Rating, Demand)
hotel('Cairo', 'Hilton Cairo Nile', 1200, 4.7, high).
hotel('Cairo', 'InterContinental Cairo Semiramis', 1300, 4.8, high).
hotel('Cairo', 'Marriott Mena House', 1400, 4.9, high).
hotel('Cairo', 'Four Seasons Nile Plaza', 1500, 4.8, high).
hotel('Cairo', 'The Nile Ritz-Carlton', 1250, 4.7, high).
hotel('Cairo', 'Sofitel Cairo Downtown Nile', 1100, 4.6, high).
hotel('Cairo', 'Budget Hotel Cairo', 800, 4.0, high).
hotel('Cairo', 'Economy Hotel Cairo', 600, 3.8, high).
hotel('Aswan', 'Sofitel Legend Old Cataract', 1600, 4.9, high).
hotel('Aswan', 'Mövenpick Resort Aswan', 1000, 4.6, high).
hotel('Aswan', 'Hilton Aswan', 1100, 4.7, high).
hotel('Aswan', 'Anakato Nubian Experience', 900, 4.5, low).
hotel('Aswan', 'Pyramisa Isis Island', 950, 4.4, high).
hotel('PortSaid', 'Resta Port Said Hotel', 800, 4.3, high).
hotel('PortSaid', 'Grand Hotel Port Said', 600, 4.0, low).
hotel('PortSaid', 'Helnan Port Said', 850, 4.4, high).
hotel('PortSaid', 'Porto Said Resort', 750, 4.2, high).
hotel('PortSaid', 'Jewel Port Said', 700, 4.1, high).
```

Figure 1: Hotel/5

2. *hotel_review/3*

- **Structure:** `hotel_review(HotelName, Rating, Comment)`
- **Description:** Stores user-submitted reviews for hotels, enhancing *decision-making* with **qualitative** feedback.
- **Arguments:**
 - HotelName: The name of the reviewed hotel (e.g., 'Hilton Cairo Nile').
 - Rating: The user's rating (0–5 scale, e.g., 4.5).
 - Comment: A textual comment describing the user's *experience* (e.g., 'Stunning Nile views').
- **Example:** `hotel_review('Hilton Cairo Nile', 4.5, 'Stunning Nile views, impeccable service')`
- **Purpose:** Supports hotel selection by providing average review ratings and user feedback, dynamically updated via `submit_hotel_review/4`.

```
% hotel_review(HotelName, Rating, Comment)
hotel_review('Hilton Cairo Nile', 4.5, 'Stunning Nile views, impeccable service').
hotel_review('InterContinental Cairo Semiramis', 4.8, 'Luxurious rooms, fantastic dining').
hotel_review('Marriott Mena House', 4.9, 'Iconic hotel with pyramid views').
hotel_review('Four Seasons Nile Plaza', 4.7, 'Elegant ambiance, top-notch spa').
hotel_review('The Nile Ritz-Carlton', 4.6, 'Central location, vibrant atmosphere').
hotel_review('Sofitel Cairo Downtown Nile', 4.5, 'Modern design, amazing rooftop bar').
hotel_review('Sofitel Legend Old Cataract', 4.9, 'Historic charm, breathtaking Nile views').
hotel_review('Movenpick Resort Aswan', 4.6, 'Serene island setting, excellent pool').
hotel_review('Hilton Aswan', 4.7, 'Comfortable rooms, friendly staff').
hotel_review('Anakato Nubian Experience', 4.4, 'Unique cultural stay, warm hospitality').
hotel_review('Pyramisa Isis Island', 4.3, 'Spacious grounds, good value').
hotel_review('Resta Port Said Hotel', 4.2, 'Clean rooms, seafront location').
hotel_review('Grand Hotel Port Said', 3.8, 'Decent budget option, good location').
hotel_review('Helnan Port Said', 4.4, 'Modern amenities, excellent service').
hotel_review('Porto Said Resort', 4.1, 'Nice beach access, family-friendly').
hotel_review('Jewel Port Said', 4.0, 'Affordable, clean, convenient').
```

Figure 2: *Hotel_review/3*

3. *transport/5*

- **Structure:** `transport(City, Mode, Cost, Distance, Duration)`
- **Description:** Lists available *transportation* modes for each city, facilitating **cost-effective** travel planning.
- **Arguments:**
 - City: The city of operation (e.g., 'Cairo').
 - Mode: The transport type (e.g., 'Metro', 'Taxi').
 - Cost: The cost in **EGP** (e.g., 10).
 - Distance: The typical distance covered in **kilometers** (e.g., 15).
 - Duration: The estimated travel *time* in **hours** (e.g., 0.5).
- **Example:** `transport('Cairo', 'Metro', 10, 15, 0.5)`

- **Purpose:** Enables selection of the cheapest transport mode suitable for the user's city and *budget*.

```
% transport(City, Mode, Cost, Distance, Duration)
transport('Cairo', 'Metro', 10, 15, 0.5).
transport('Cairo', 'Taxi', 50, 15, 0.75).
transport('Cairo', 'Bus', 8, 20, 1.0).
transport('Cairo', 'Train', 40, 200, 2.5).
transport('Aswan', 'Felucca', 100, 5, 1.5).
transport('Aswan', 'Minibus', 15, 10, 0.5).
transport('Aswan', 'Taxi', 30, 10, 0.4).
transport('Aswan', 'Train', 60, 300, 4.0).
transport('PortSaid', 'Ferry', 20, 10, 0.6).
transport('PortSaid', 'Taxi', 25, 8, 0.3).
transport('PortSaid', 'Bus', 12, 15, 0.7).
transport('PortSaid', 'Minibus', 10, 8, 0.4).
```

Figure 3: transport/5

4. activity/6

- **Structure:** `activity(City, Name, Cost, Duration, Category, Suitability)`
- **Description:** Enumerates activities available in each city, supporting *itinerary* planning with **diverse** options.
- **Arguments:**
 - City: The city where the activity is available (e.g., 'Cairo').
 - Name: The activity name (e.g., 'Pyramid Tour').
 - Cost: The cost in EGP (e.g., 500).
 - Duration: The duration in hours (e.g., 4).
 - Category: The activity type (e.g., 'cultural', 'leisure', 'adventure').
 - Suitability: The target audience (e.g., 'group', 'solo', 'family').
- **Example:** `activity('Cairo', 'Pyramid Tour', 500, 4, cultural, group)`
- **Purpose:** Provides data for activity selection and distribution, ensuring activities align with *time*, *budget*, and user preferences.

```
% activity(City, Name, Cost, Duration, Category, Suitability)
activity('Cairo', 'Pyramid Tour', 500, 4, cultural, group).
activity('Cairo', 'Egyptian Museum Visit', 200, 3, cultural, solo).
activity('Cairo', 'Nile River Cruise', 300, 2, leisure, family).
activity('Cairo', 'Desert Quad Biking', 600, 3, adventure, group).
activity('Cairo', 'Khan el-Khalili Bazaar', 100, 2, cultural, solo).
activity('Cairo', 'Citadel and Mohamed Ali Mosque Tour', 350, 3, cultural, group).
activity('Cairo', 'Sound and Light Show at Pyramids', 250, 1.5, leisure, family).
activity('Cairo', 'Camel Ride at Giza', 200, 1, adventure, solo).
activity('Cairo', 'Coptic Cairo Walking Tour', 150, 2.5, cultural, group).
activity('Cairo', 'Street Food Tour', 100, 2, leisure, solo).
activity('Aswan', 'Philae Temple Tour', 400, 3, cultural, group).
activity('Aswan', 'Felucca Sunset Sail', 150, 1.5, leisure, family).
activity('Aswan', 'Nubian Village Visit', 250, 4, cultural, solo).
activity('Aswan', 'High Dam Tour', 200, 2, cultural, group).
activity('Aswan', 'Sandboarding Adventure', 350, 3, adventure, group).
activity('Aswan', 'Abu Simbel Day Trip', 800, 8, cultural, group).
activity('Aswan', 'Botanical Garden Visit', 100, 1.5, leisure, family).
activity('Aswan', 'Nile Kayaking', 300, 2, adventure, solo).
activity('Aswan', 'Aswan Market Tour', 120, 2, cultural, solo).
activity('Aswan', 'Stargazing in the Desert', 400, 4, leisure, group).
activity('PortSaid', 'Suez Canal Boat Tour', 300, 2.5, cultural, group).
activity('PortSaid', 'Beach Relaxation', 50, 4, leisure, family).
activity('PortSaid', 'PortSaid Museum Visit', 100, 2, cultural, solo).
activity('PortSaid', 'Water Sports', 400, 3, adventure, group).
activity('PortSaid', 'Coastal Bike Tour', 150, 2, leisure, solo).
activity('PortSaid', 'Fishing Trip', 250, 3, adventure, group).
activity('PortSaid', 'Corniche Evening Walk', 50, 1.5, leisure, family).
activity('PortSaid', 'Military Museum Visit', 80, 1.5, cultural, solo).
activity('PortSaid', 'Kite Surfing Lesson', 500, 2, adventure, solo).
activity('PortSaid', 'Local Cafe Hopping', 100, 2, leisure, group).
```

Figure 4: activity/6

5. attraction/4

- **Structure:** attraction(City, Name, Type, VisitingHours)
- **Description:** Details landmarks and *cultural* sites, linked to activities for enhanced **destination** insights.
- **Arguments:**
 - City: The city of the attraction (e.g., 'Cairo').
 - Name: The attraction name (e.g., 'Giza Pyramids').
 - Type: The attraction type ('landmark', 'cultural_site', 'tourist_spot').
 - VisitingHours: A list of opening and closing hours (e.g., [8, 17]).
- **Example:** attraction('Cairo', 'Giza Pyramids', landmark, [8, 17])
- **Purpose:** Enriches activity planning by linking activities to **attractions**, providing users with visiting hours and type information.


```
% attraction(City, Name, Type, VisitingHours[Open, Close])
attraction('Cairo', 'Giza Pyramids', landmark, [8, 17]).
attraction('Cairo', 'Egyptian Museum', cultural_site, [9, 17]).
attraction('Cairo', 'Khan el-Khalili', tourist_spot, [9, 22]).
attraction('Cairo', 'Citadel of Saladin', cultural_site, [8, 16]).
attraction('Cairo', 'Coptic Cairo', cultural_site, [9, 17]).
attraction('Aswan', 'Philae Temple', cultural_site, [7, 16]).
attraction('Aswan', 'Abu Simbel', landmark, [6, 15]). % Often visited from Aswan
attraction('Aswan', 'Nubian Village', tourist_spot, [8, 18]).
attraction('Aswan', 'Aswan High Dam', landmark, [8, 16]).
attraction('Aswan', 'Botanical Garden', tourist_spot, [8, 17]).
attraction('PortSaid', 'Suez Canal', landmark, [7, 17]).
attraction('PortSaid', 'PortSaid Military Museum', cultural_site, [9, 15]).
attraction('PortSaid', 'PortSaid Corniche', tourist_spot, [0, 24]). % Assuming always accessib
attraction('PortSaid', 'Al-Nasr Museum', cultural_site, [9, 16]).
attraction('PortSaid', 'Ferial Garden', tourist_spot, [8, 20]).
```

Figure 5: attraction/4

6. daily_time_constraint/1

- **Structure:** `daily_time_constraint(MaxHoursPerDay)`
- **Description:** Defines valid maximum hours per day for activities, ensuring *realistic scheduling*.
- **Arguments:**
 - MaxHoursPerDay: Allowed hours (e.g., 4, 6, 8, 10).
- **Example:** `daily_time_constraint(8)`
- **Purpose:** Constrains activity distribution to user-specified daily *time* limits.

```
% daily_time_constraint(MaxHoursPerDay) - Predefined valid values
daily_time_constraint(4).
daily_time_constraint(6).
daily_time_constraint(8).
daily_time_constraint(10).
```

Figure 6: daily_time_constraint/1

7. food_cost/2

- **Structure:** `food_cost(City, DailyCost)`
- **Description:** Provides tiered daily food cost estimates for each city, supporting *budget* planning.
- **Arguments:**
 - City: The city (e.g., 'Cairo').
 - DailyCost: The cost in EGP for low, mid, or high budgets (e.g., 250).
- **Example:** `food_cost('Cairo', 250)`
- **Purpose:** Enables *budget-aware* food cost selection, aligning with the user's **financial** constraints.

```
% food_cost(City, DailyCost) - Different budget levels
food_cost('Cairo', 250). % Low
food_cost('Cairo', 400). % Mid
food_cost('Cairo', 600). % High
food_cost('Aswan', 120). % Low
food_cost('Aswan', 250). % Mid
food_cost('Aswan', 380). % High
food_cost('PortSaid', 150). % Low
food_cost('PortSaid', 280). % Mid
food_cost('PortSaid', 450). % High
```

Figure 7: food_cost/2

8. month_to_number/2

- **Structure:** month_to_number(Month, Number)
- **Description:** Maps month names to numerical values for *seasonal* validation.
- **Arguments:**
 - Month: The month name (e.g., 'January').
 - Number: The corresponding number (1–12, e.g., 1).
- **Example:** month_to_number('January', 1)
- **Purpose:** Supports **seasonal** checks by converting user-input months to numbers.

```
% month_to_number(Month, Number)
month_to_number('January', 1).
month_to_number('February', 2).
month_to_number('March', 3).
month_to_number('April', 4).
month_to_number('May', 5).
month_to_number('June', 6).
month_to_number('July', 7).
month_to_number('August', 8).
month_to_number('September', 9).
month_to_number('October', 10).
month_to_number('November', 11).
month_to_number('December', 12).
```

Figure 8: month_to_number/2

9. season/2

- **Structure:** season(Month, Season)
- **Description:** Associates months with seasons for *travel* suitability checks.

- **Arguments:**
 - Month: The month name (e.g., 'January').
 - Season: The season ('winter', 'spring', 'summer', 'autumn').
- **Example:** `season('January', winter)`
- **Purpose:** Facilitates **season-based** validation of *travel* plans.

```
% season(Month, Season)
season('January', winter).
season('February', winter).
season('March', spring).
season('April', spring).
season('May', spring).
season('June', summer).
season('July', summer).
season('August', summer).
season('September', autumn).
season('October', autumn).
season('November', autumn).
season('December', winter).
```

Figure 9: season/2

10.city/3

- **Structure:** `city(Name, BestSeason, Currency)`
- **Description:** Defines cities, their **optimal** *travel* seasons, and currency, guiding **destination** selection.
- **Arguments:**
 - Name: The city name (e.g., 'Cairo').
 - BestSeason: The recommended season ('spring', 'autumn', etc.).
 - Currency: The currency used (e.g., 'EGP').
- **Example:** `city('Cairo', spring, 'EGP')`
- **Purpose:** Ensures city-specific planning aligns with **optimal** *travel* seasons and currency.

```
% city(Name, BestSeason, Currency)
city('Cairo', spring, 'EGP').
city('Cairo', autumn, 'EGP').
city('Aswan', winter, 'EGP').
city('Aswan', spring, 'EGP').
city('PortSaid', summer, 'EGP').
city('PortSaid', spring, 'EGP').
```

Figure 10: city/3

11.activity_attraction_link/2

- **Structure:** `activity_attraction_link(ActivityName, AttractionName)`
- **Description:** Links activities to specific **attractions**, enhancing *itinerary* context.
- **Arguments:**
 - ActivityName: The activity name (e.g., 'Pyramid Tour').
 - AttractionName: The linked attraction (e.g., 'Giza Pyramids').
- **Example:** `activity_attraction_link('Pyramid Tour', 'Giza Pyramids')`
- **Purpose:** Provides context for activities by associating them with **attractions**, displayed in the *travel* plan.

```
% activity_attraction_link(ActivityName, AttractionName) - Explicit links
activity_attraction_link('Pyramid Tour', 'Giza Pyramids').
activity_attraction_link('Egyptian Museum Visit', 'Egyptian Museum').
activity_attraction_link('Khan el-Khalili Bazaar', 'Khan el-Khalili').
activity_attraction_link('Citadel and Mohamed Ali Mosque Tour', 'Citadel of Saladin').
activity_attraction_link('Coptic Cairo Walking Tour', 'Coptic Cairo').
activity_attraction_link('Camel Ride at Giza', 'Giza Pyramids').
activity_attraction_link('Desert Quad Biking', 'Giza Pyramids'). % Assuming near Giza
activity_attraction_link('Philae Temple Tour', 'Philae Temple').
activity_attraction_link('Nubian Village Visit', 'Nubian Village').
activity_attraction_link('High Dam Tour', 'Aswan High Dam').
activity_attraction_link('Abu Simbel Day Trip', 'Abu Simbel').
activity_attraction_link('Botanical Garden Visit', 'Botanical Garden').
activity_attraction_link('Suez Canal Boat Tour', 'Suez Canal').
activity_attraction_link('PortSaid Museum Visit', 'Al-Nasr Museum'). % Assuming this is the main one
activity_attraction_link('Military Museum Visit', 'PortSaid Military Museum').
activity_attraction_link('Corniche Evening Walk', 'PortSaid Corniche').
```

Figure 11: activity_attraction_link/2

Rules

The rules (predicates) define the *logic* for processing user inputs, selecting **travel** components, and generating **itineraries**. Each rule is described with its functionality, arguments, and role in the system.

1. *start_planner/0*

- **Description:** Initiates the **interactive command-line interface (CLI)** for the travel planner, guiding users through input collection.
- **Functionality:**
 - Prompts for user inputs: name (UserID), **destination** city, *trip* duration, *travel* month, maximum activity hours per day, group size, and **budget** range (minimum and maximum).
 - Validates inputs using helper predicates and calls *plan_trip/8* to generate the **travel plan**.
 - Catches and displays errors, ensuring a **user-friendly experience**.
- **Example Usage:** ?- *start_planner*.
- **Purpose:** Serves as the **entry point** for users, orchestrating the planning process with validated inputs.

```
start_planner :-
    write('--- Welcome to the Travel Planner ---'), nl,
    prompt_read_string('Enter your name (UserID): ', UserID),
    prompt_read_city('Enter destination city (Cairo, Aswan, PortSaid): ', City),
    prompt_read_integer('Enter trip duration in days: ', DurationDays),
    prompt_read_month('Enter travel month (e.g., January, February): ', Month),
    prompt_read_hours('Enter max activity hours per day (4, 6, 8, 10): ', MaxHoursPerDay),
    prompt_read_integer('Enter group size: ', GroupSize),
    prompt_read_number('Enter minimum budget (EGP): ', MinBudget),
    prompt_read_max_budget('Enter maximum budget (EGP): ', MinBudget, MaxBudget),
    nl, write('--- Generating Plan... ---'), nl, nl,
    % Call the main planner, catch errors locally
    catch(
        plan_trip(UserID, City, DurationDays, Month, MaxHoursPerDay, GroupSize, MinBudget, MaxBudget),
        Error,
        (
            write('Error during planning: '), writeln(Error),
            write('Failed to generate a complete plan. Please check inputs or adjust constraints.'), nl
        )
    ),
    nl, write('--- Planner Finished ---'), nl.
```

Figure 12: *start_planner*.

2. *validate_inputs/5*

- **Structure:** *validate_inputs*(DurationDays, MaxHoursPerDay, GroupSize, MinBudget, MaxBudget)
- **Description:** Ensures all user inputs meet system requirements to prevent errors during planning.
- **Arguments:**
 - DurationDays: *Trip* duration in days (positive integer).
 - MaxHoursPerDay: Maximum activity hours per day (4, 6, 8, or 10).
 - GroupSize: Number of travelers (positive integer).

- MinBudget: Minimum **budget** in EGP (non-negative number).
- MaxBudget: Maximum **budget** in EGP ($\geq$ MinBudget).
- **Functionality:**
 - Checks that DurationDays and GroupSize are positive integers.
 - Verifies MaxHoursPerDay is in daily_time_constraint/1.
 - Ensures MinBudget is non-negative and MaxBudget $\geq$ MinBudget.
 - Throws errors with **descriptive** messages if validation fails.
- **Example:** validate_inputs(3, 8, 4, 5000, 10000)
- **Purpose:** Guarantees **robust** input data for *reliable* plan generation.

```
% validate_inputs(DurationDays, MaxHoursPerDay, GroupSize, MinBudget, MaxBudget)
% Validates inputs (internal check).
validate_inputs(DurationDays, MaxHoursPerDay, GroupSize, MinBudget, MaxBudget) :-
    (integer(DurationDays), DurationDays > 0 -> true ; throw(error(invalid_duration_days, context('DurationDays must be a positive integer')))),
    (daily_time_constraint(MaxHoursPerDay) -> true ; throw(error(invalid_max_hours, context('MaxHoursPerDay must be 4, 6, 8, or 10')))),
    (integer(GroupSize), GroupSize > 0 -> true ; throw(error(invalid_group_size, context('GroupSize must be positive integer')))),
    (number(MinBudget), MinBudget >= 0 -> true ; throw(error(invalid_min_budget, context('MinBudget must be non-negative')))),
    (number(MaxBudget), MaxBudget >= MinBudget -> true ; throw(error(invalid_max_budget, context('MaxBudget must be greater than or equal to MinBudget')))).
```

Figure 13: validate_inputs/5

3. check_season/3

- **Structure:** check_season(City, Month, Season)
- **Description:** Validates the suitability of the *travel* month for the chosen city based on **optimal** seasons.
- **Arguments:**
 - City: The **destination** city (e.g., 'Cairo').
 - Month: The *travel* month (e.g., 'April').
 - Season: The corresponding season (e.g., 'spring').
- **Functionality:**
 - Uses month_to_number/2 and season/2 to determine the season.
 - Checks if the season matches the city's best seasons in city/3.
 - Issues a warning with **recommended** seasons if the month is suboptimal but allows planning to proceed.
- **Example:** check_season('Cairo', 'April', spring)
- **Purpose:** Enhances plan quality by advising on **optimal** *travel* timing.

```
% check_season(City, Month, Season)
% Checks travel suitability based on predefined best seasons.
check_season(City, Month, Season) :-
    month_to_number(Month, _Number), % Fails if Month is invalid
    season(Month, Season),
    ( city(City, Season, _Currency) ->
        true % It's a good season
    ;   format('Warning: ~w might not be the best season for ~w. Recommended seasons are: ', [Month, City],
        findall(S, city(City, S, _), RecommendedSeasons),
        atomic_list_concat(RecommendedSeasons, ', ', RecStr),
        format('~w.~n', [RecStr]) % Allow planning but warn
    ).
```

Figure 14: check_season/3

4. select_hotel/7

- **Structure:** select_hotel(City, DurationDays, MinBudget, MaxBudget, HotelName, PricePerNight, Rating)
- **Description:** Selects a hotel that fits the user's **budget** and preferences, prioritizing **quality** and *cost*.
- **Arguments:**
 - City: The **destination** city.
 - DurationDays: *Trip* duration in days.
 - MinBudget: Minimum total **budget**.
 - MaxBudget: Maximum total **budget**.
 - HotelName: Selected hotel name (output).
 - PricePerNight: Hotel's cost per night (output).
 - Rating: Hotel's rating (output).
- **Functionality:**
 - Calculates hotel **budget** as 20–35% of total **budget** (MinHotelBudget, MaxHotelBudget).
 - Collects hotels from hotel/5 where total cost (**PricePerNight** * **DurationDays**) fits the **budget**.
 - Computes average review ratings from hotel_review/3 or uses base rating if no reviews.
 - Sorts hotels by average review rating (descending) and price (ascending), selecting the best.
 - Throws an error if no hotels are available within **budget**.
- **Example:** select_hotel('Cairo', 3, 5000, 10000, 'Hilton Cairo Nile', 1200, 4.7)
- **Purpose:** Ensures **cost-effective**, **high-quality accommodation** selection.

```
% select_hotel(City, DurationDays, MinBudget, MaxBudget, HotelName, PricePerNight, Rating)
% Selects a hotel within budget, prioritizing rating then price.
select_hotel(City, DurationDays, MinBudget, MaxBudget, HotelName, PricePerNight, Rating) :-
    MinHotelBudget is MinBudget * 0.20, % Min 20% of total min budget for hotel
    MaxHotelBudget is MaxBudget * 0.35, % Max 35% of total max budget for hotel
    findall(AvgReviewRating-Price-HotelRating-Hotel, (
        hotel(City, Hotel, Price, HotelRating, _),
        number(Price), Price > 0,
        TotalHotelCost is Price * DurationDays,
        TotalHotelCost <= MaxHotelBudget,
        TotalHotelCost >= MinHotelBudget,
        findall(ReviewRating, hotel_review(Hotel, ReviewRating, _), ReviewRatings),
        ( ReviewRatings = [] -> AvgReviewRating is HotelRating ; % Use base rating if no reviews
          sum_list(ReviewRatings, Sum), length(ReviewRatings, Len), AvgReviewRating is Sum / Len
        )
    ), Hotels),
    ( Hotels = [] ->
        format(atom(ErrorMessage), 'No hotels available in ~w within budget range for hotel (~1f - ~1f EGP for ~w days)',
            [City, MinHotelBudget, MaxHotelBudget, DurationDays]),
        throw(error(no_hotels_available, context(ErrorMessage)))
    ); true
),
sort(1, @>=, Hotels, SortedByAvgRating), % Sort by AvgReviewRating (descending)
sort(2, @<=, SortedByAvgRating, SortedHotels), % Then sort by Price (ascending)
SortedHotels = [BestAvgRating-PricePerNight-Rating-HotelName | _], % Select the best
number(PricePerNight), number(Rating), number(BestAvgRating), % Ensure numeric
!.
```

Figure 15: select_hotel/7

5. select_transport/3

- **Structure:** select_transport(City, TransportMode, TransportCost)
- **Description:** Chooses the **cheapest** available *transport* mode for the specified city.
- **Arguments:**
 - City: The **destination** city.
 - TransportMode: Selected *transport* mode (output).
 - TransportCost: Cost in EGP (output).
- **Functionality:**
 - Collects all *transport* options from transport/5 for the city.
 - Sorts by cost (ascending) and selects the **cheapest**.
 - Throws an error if no *transport* is available.
- **Example:** select_transport('Cairo', 'Metro', 10)
- **Purpose:** Optimizes **transportation** costs for the *travel* plan.

```
% select_transport(City, TransportMode, TransportCost)
% Selects cheapest transport.
select_transport(City, TransportMode, TransportCost) :-
    findall(Cost-Mode, (
        transport(City, Mode, Cost, _, _),
        number(Cost), Cost >= 0 % Ensure numeric and non-negative cost
    ), Transports),
    ( Transports = [] ->
        format(atom(ErrorMessage), 'No transport available in ~w', [City]),
        throw(error(no_transport_available, context(ErrorMessage)))
    ); true
),
sort(1, @<=, Transports, [TransportCost-TransportMode|_]), % Sort by cost ascending, take first
number(TransportCost). % Ensure numeric
```

Figure 16: select_transport/3

6. *select_food_cost/4*

- **Structure:** `select_food_cost(City, MinBudget, MaxBudget, DailyFoodCost)`
- **Description:** Selects a daily food cost based on the **budget**, preferring lower costs for **modest** budgets.
- **Arguments:**
 - City: The **destination** city.
 - MinBudget: Minimum total **budget**.
 - MaxBudget: Maximum total **budget**.
 - DailyFoodCost: Selected daily food cost (output).
- **Functionality:**
 - Collects food costs from `food_cost/2` for the city.
 - Sorts costs ascending and selects based on **budget** tiers:
 - Low cost for budgets < 4000 EGP (midpoint).
 - Mid cost for budgets 4000–8000 EGP.
 - High cost for budgets > 8000 EGP.
 - Handles cases with fewer cost options (e.g., *only low or low/mid*).
 - Throws an error if no food costs are defined.
- **Example:** `select_food_cost('Cairo', 5000, 10000, 400)`
- **Purpose:** Aligns food expenses with the user's **financial** constraints.

```
% select_food_cost(City, MinBudget, MaxBudget, DailyFoodCost)
% Selects lowest food cost that potentially fits within budget.
select_food_cost(City, MinBudget, MaxBudget, DailyFoodCost) :-
    findall(Cost, (
        food_cost(City, Cost),
        number(Cost), Cost >= 0
    ), Costs),
    ( Costs = [] ->
        format(atom(ErrorMessage), 'No food costs defined for ~w', [City]),
        throw(error(no_food_costs, context(ErrorMessage)))
    ; true
    ),
    sort(Costs, SortedCosts), % Sort ascending
    % Basic check: select the cheapest unless the budget is very high
    MidBudget is (MinBudget + MaxBudget) / 2,
    length(SortedCosts, NumLevels),
    ( NumLevels == 0 -> throw(error(no_food_costs, context(City))) ; true ), % Should not happen if Costs check passed
    ( NumLevels > 2 -> % If Low, Mid, High exist
        nth1(1, SortedCosts, LowCost),
        nth1(2, SortedCosts, MidCost),
        nth1(3, SortedCosts, HighCost),
        ( MidBudget > 8000 -> DailyFoodCost = HighCost % High budget -> high food cost
        ; MidBudget > 4000 -> DailyFoodCost = MidCost % Mid budget -> mid food cost
        ; DailyFoodCost = LowCost % Default to lowest
        )
    ; NumLevels == 2 -> % Only Low, Mid (or similar)
        nth1(1, SortedCosts, LowCost),
        nth1(2, SortedCosts, HighCost), % Treat second as 'high'
        ( MidBudget > 5000 -> DailyFoodCost = HighCost
        ; DailyFoodCost = LowCost
        )
    ; NumLevels == 1 -> % Only one option
        SortedCosts = [DailyFoodCost|_ ]
    ).
```

Figure 17: *select_food_cost/4*

7. *collect_all_activities/2*

- **Structure:** `collect_all_activities(City, ActivitiesRaw)`
- **Description:** Gathers all **available** activities in the specified city for *itinerary* planning.
- **Arguments:**
 - City: The **destination** city.
 - ActivitiesRaw: List of tuples [Cost-Duration-Activity] (output).
- **Functionality:**
 - Queries activity/6 to collect activities, ensuring **valid** costs (≥ 0) and durations (> 0).
 - Returns a list of Cost-Duration-Activity tuples.
 - Throws an error if no activities are **available**.
- **Example:** `collect_all_activities('Cairo', [500-4-'Pyramid Tour', 200-3-'Egyptian Museum Visit'])`
- **Purpose:** Provides the pool of activities for distribution across the *trip*.

```
% collect_all_activities(City, ActivitiesRaw)
% Collects all activities in the city as Cost-Duration-Name tuples.
collect_all_activities(City, ActivitiesRaw) :-
    findall(Cost-Duration-Activity, (
        activity(City, Activity, Cost, Duration, _Category, _Suitability),
        number(Cost), Cost >= 0, % Ensure numeric cost
        number(Duration), Duration > 0 % Ensure numeric duration
    ), ActivitiesRaw),
    ( ActivitiesRaw = [] ->
        format(atom(ErrorMessage), 'No activities available in ~w', [City]),
        throw(error(no_activities_available, context(ErrorMessage)))
    ; true
    ).
```

Figure 18: *collect_all_activities/2*

8. *distribute_activities/8*

- **Structure:** `distribute_activities(DurationDays, MaxHoursPerDay, MaxBudget, MinBudget, PricePerNight, DailyFoodCost, TransportCost, AllActivitiesRaw, DailyActivitiesRaw, ActivityCosts, DailyActivityDetails)`
- **Description:** Allocates activities across *trip* days, respecting **time** and **budget** constraints.
- **Arguments:**
 - DurationDays: *Trip* duration in days.
 - MaxHoursPerDay: Maximum activity hours per day.
 - MaxBudget: Maximum total **budget**.
 - MinBudget: Minimum total **budget**.
 - PricePerNight: Hotel cost per night.
 - DailyFoodCost: Daily food cost.

- TransportCost: *Transport* cost.
 - AllActivitiesRaw: List of all activities ([Cost-Duration-Activity]).
 - DailyActivitiesRaw: List of daily activity tuples (output).
 - ActivityCosts: List of daily activity costs (output).
 - DailyActivityDetails: List of daily activity details (output).
- **Functionality:**
 - Calculates **fixed** costs ($\text{PricePerNight} * \text{DurationDays} + \text{DailyFoodCost} * \text{DurationDays} + \text{TransportCost}$).
 - Determines remaining **budget** for activities (MaxActivityBudget, MinActivityBudget).
 - Computes average daily activity **budget**.
 - Calls `distribute_activities_by_day/10` to assign activities.
 - Reverses output lists for **correct** day order.
 - Throws errors for **invalid** inputs (e.g., non-numeric costs).
 - **Example:** Distributes activities for a 3-day *trip*, ensuring daily costs and hours fit constraints.
 - **Purpose:** Creates a **balanced itinerary** within **time** and **budget** limits.

```
% distribute_activities(DurationDays, MaxHoursPerDay, MaxBudget, MinBudget, PricePerNight, DailyFoodCost, TransportCost, AllActivitiesRaw,
% DailyActivitiesRaw, ActivityCosts, DailyActivityDetails)
% Distributes activities across days, respecting time and budget.
distribute_activities(DurationDays, MaxHoursPerDay, MaxBudget, MinBudget, PricePerNight, DailyFoodCost, TransportCost, AllActivitiesRaw,
DailyActivitiesRaw, ActivityCosts, DailyActivityDetails) :-
    % Calculate fixed costs
    ( number(PricePerNight), number(DailyFoodCost), number(TransportCost), integer(DurationDays) ->
      TotalFixedCost is PricePerNight * DurationDays + DailyFoodCost * DurationDays + TransportCost,
      % Budget remaining for activities
      MaxActivityBudget is max(0, MaxBudget - TotalFixedCost), % Ensure non-negative
      MinActivityBudget is max(0, MinBudget - TotalFixedCost), % Ensure non-negative

      % Average daily budget for activities
      AvgMaxDailyActivityCost is MaxActivityBudget / DurationDays,
      AvgMinDailyActivityCost is MinActivityBudget / DurationDays,

      % Distribute activities trying to balance cost and time
      sort(1, @=<, AllActivitiesRaw, SortedActivitiesByCost), % Sort by cost ascending for selection
      distribute_activities_by_day(DurationDays, MaxHoursPerDay, AvgMaxDailyActivityCost, AvgMinDailyActivityCost, SortedActivitiesByCost,
      [], [], [], DailyActivitiesRawRev, ActivityCostsRev, DailyActivityDetailsRev),

      % Reverse lists to get correct day order
      reverse(DailyActivitiesRawRev, DailyActivitiesRaw),
      reverse(ActivityCostsRev, ActivityCosts),
      reverse(DailyActivityDetailsRev, DailyActivityDetails)

    ; format(atom(ErrorMessage), 'Invalid input to distribute_activities: PricePerNight=~w, DailyFoodCost=~w, TransportCost=~w, DurationDays=~w', [Price
PerNight, DailyFoodCost, TransportCost, DurationDays]),
      throw(error(invalid_input, context(ErrorMessage)))
    ).
```

Figure 19: `collect_all_activities/2`

9. `distribute_activities_by_day/10`

- **Structure:** `distribute_activities_by_day(DaysLeft, MaxHoursPerDay, AvgMaxDailyCost, AvgMinDailyCost, AvailableActivities, AccActivitiesRaw, AccCosts, AccDetails, FinalActivitiesRaw, FinalCosts, FinalDetails)`
- **Description:** Recursively allocates activities to each day, ensuring daily constraints are met.
- **Arguments:**
 - DaysLeft: Remaining days to plan.
 - MaxHoursPerDay: Maximum activity hours per day.
 - AvgMaxDailyCost: Average maximum daily activity cost.

- AvgMinDailyCost: Average minimum daily activity cost.
 - AvailableActivities: Remaining activities ([Cost-Duration-Activity]).
 - AccActivitiesRaw: Accumulated daily activity tuples.
 - AccCosts: Accumulated daily costs.
 - AccDetails: Accumulated daily details.
 - FinalActivitiesRaw, FinalCosts, FinalDetails: **Final** outputs.
- **Functionality:**
 - Base case: Stops when DaysLeft = 0, returning accumulated lists.
 - Recursive case: Calls select_daily_activities/6 to select activities for the **current** day, subtracts them from **available** activities, and recurses for the next day.
 - **Example:** Allocates activities for each day, e.g., Day 1: ['Pyramid Tour'], Day 2: ['Egyptian Museum Visit'].
 - **Purpose:** Ensures **systematic** activity distribution across all *trip* days.

```
% distribute_activities_by_day(DaysLeft, MaxHoursPerDay, AvgMaxDailyCost, AvgMinDailyCost, AvailableActivities,
% AccActivitiesRaw, AccCosts, AccDetails, FinalActivitiesRaw, FinalCosts, FinalDetails)
% Recursive helper for activity distribution.
distribute_activities_by_day(0, _, _, _, AccActivitiesRaw, AccCosts, AccDetails, AccActivitiesRaw, AccCosts, AccDetails). % Base case

distribute_activities_by_day(DaysLeft, MaxHoursPerDay, AvgMaxDailyCost, AvgMinDailyCost, AvailableActivities,
AccActivitiesRaw, AccCosts, AccDetails, FinalActivitiesRaw, FinalCosts, FinalDetails) :-
    DaysLeft > 0,
    % Select activities for the current day
    select_daily_activities(AvailableActivities, MaxHoursPerDay, AvgMaxDailyCost, AvgMinDailyCost,
DayActivitiesRaw, DayCost, DayDetails),

    % Update remaining activities (subtract selected ones)
    subtract(AvailableActivities, DayActivitiesRaw, RemainingActivities),

    % Recurse for the next day
    NextDaysLeft is DaysLeft - 1,
    distribute_activities_by_day(NextDaysLeft, MaxHoursPerDay, AvgMaxDailyCost, AvgMinDailyCost, RemainingActivities,
[DayActivitiesRaw | AccActivitiesRaw], [DayCost | AccCosts], [DayDetails | AccDetails],
FinalActivitiesRaw, FinalCosts, FinalDetails).
```

Figure 20: distribute_activities_by_day/10

10.select_daily_activities/6

- **Structure:** select_daily_activities(AvailableActivities, MaxHours, MaxCost, MinCost, SelectedActivitiesRaw, TotalCost, SelectedDetails)
- **Description:** Selects activities for a single day, prioritizing those within **time** and **budget** limits.
- **Arguments:**
 - AvailableActivities: List of **available** activities ([Cost-Duration-Activity]).
 - MaxHours: Maximum hours for the day.
 - MaxCost: Maximum cost for the day.
 - MinCost: Minimum cost for the day (optional enforcement).
 - SelectedActivitiesRaw: Selected activity tuples (output).
 - TotalCost: Total cost of selected activities (output).
 - SelectedDetails: Details of selected activities (output).

- **Functionality:**
 - Uses a *greedy* approach via `select_activities_greedy/9` to pick **cheapest** activities fitting `MaxHours` and `MaxCost`.
 - Reverses selected activities for **correct** order.
 - Extracts details as `ActivityName-Cost-Duration` tuples.
 - Returns empty lists with a warning if no activities fit.
- **Example:** `select_daily_activities([500-4-'Pyramid Tour', 200-3-'Egyptian Museum Visit'], 6, 500, 0, [200-3-'Egyptian Museum Visit'], 200, ['Egyptian Museum Visit'-200-3])`
- **Purpose:** Optimizes daily activity selection for **cost** and **time** efficiency.

```
% select_daily_activities(AvailableActivities, MaxHours, MaxCost, MinCost, SelectedActivitiesRaw, TotalCost, SelectedDetails)
% Selects activities for one day within constraints. Prioritizes fitting within MaxHours and MaxCost.
select_daily_activities(AvailableActivities, MaxHours, MaxCost, MinCost, SelectedActivitiesRaw, TotalCost, SelectedDetails) :-
    % Use greedy approach: pick cheapest first until limits hit.
    select_activities_greedy(AvailableActivities, MaxHours, MaxCost, [], 0, 0, SelectedActivitiesRev, TotalCost, _TotalHours),
    reverse(SelectedActivitiesRev, SelectedActivitiesRaw), % Correct order
    % Ensure minimum cost is met if possible, otherwise it's okay if TotalCost is 0
    ( TotalCost >= MinCost -> true ; true ), % Accept if below MinCost, including 0
    % Extract details (ensure this handles the empty list case correctly)
    ( SelectedActivitiesRaw == [] -> SelectedDetails = []
    ; maplist(extract_activity_details_tuple, SelectedActivitiesRaw, SelectedDetails) % Use tuple helper
    ),
    !. % Commit to the first solution found by greedy

% If greedy finds nothing, return empty lists
select_daily_activities(_, _, _, _, [], 0, []) :-
    format('Warning: No activities selected for a day. Constraints might be too tight.-n').
```

Figure 21: `select_daily_activities/6`

11. `calculate_total_cost/5`

- **Structure:** `calculate_total_cost(DurationDays, PricePerNight, DailyFoodCost, TransportCost, ActivityCosts, TotalBudget)`
- **Description:** Computes the **total trip** cost by summing all components.
- **Arguments:**
 - `DurationDays`: *Trip* duration in days.
 - `PricePerNight`: Hotel cost per night.
 - `DailyFoodCost`: Daily food cost.
 - `TransportCost`: *Transport* cost.
 - `ActivityCosts`: List of daily activity costs.
 - `TotalBudget`: **Total** cost (output).
- **Functionality:**
 - Calculates `TotalHotelCost = PricePerNight * DurationDays`.
 - Calculates `TotalFoodCost = DailyFoodCost * DurationDays`.
 - Sums `ActivityCosts` (0 if empty).
 - Computes `TotalBudget = TotalHotelCost + TotalFoodCost + TransportCost + TotalActivityCost`.
- **Example:** `calculate_total_cost(3, 1200, 400, 10, [500, 200], 4310)`

- **Purpose:** Provides an **accurate** cost estimate for *budget* validation.

```
% calculate_total_cost(DurationDays, PricePerNight, DailyFoodCost, TransportCost, ActivityCosts, TotalBudget)
% Calculates total cost, handling empty ActivityCosts.
calculate_total_cost(DurationDays, PricePerNight, DailyFoodCost, TransportCost, ActivityCosts, TotalBudget) :-
    TotalHotelCost is PricePerNight * DurationDays,
    TotalFoodCost is DailyFoodCost * DurationDays,
    ( ActivityCosts = [] -> TotalActivityCost = 0 ; sum_list(ActivityCosts, TotalActivityCost) ),
    TotalBudget is TotalHotelCost + TotalActivityCost + TransportCost + TotalFoodCost.
```

Figure 22: calculate_total_cost/5

12.check_budget/4

- **Structure:** `check_budget(TotalBudget, MaxBudget, MinBudget, BudgetStatus)`
- **Description:** Evaluates the **total** cost against **budget** constraints, setting an **appropriate** status.
- **Arguments:**
 - TotalBudget: Calculated **total** cost.
 - MaxBudget: Maximum allowed **budget**.
 - MinBudget: Minimum desired **budget**.
 - BudgetStatus: 'ok', 'over', or 'under' (output).
- **Functionality:**
 - Sets BudgetStatus = ok if TotalBudget is within 80% of MinBudget and $\leq$ MaxBudget.
 - Sets BudgetStatus = over and throws an error if `TotalBudget > MaxBudget`.
 - Sets BudgetStatus = under with a warning if `TotalBudget < MinBudget * 0.8`.
- **Example:** `check_budget(8000, 10000, 5000, ok)`
- **Purpose:** Ensures the plan adheres to **financial** constraints and provides feedback.

```
% check_budget(TotalBudget, MaxBudget, MinBudget, BudgetStatus)
% Checks budget and sets status. Uses 80% of MinBudget as a soft lower bound for 'ok'.
check_budget(TotalBudget, MaxBudget, MinBudget, ok) :-
    TotalBudget <= MaxBudget,
    TotalBudget >= MinBudget * 0.8, % Allow slightly under MinBudget
    !.
check_budget(TotalBudget, MaxBudget, MinBudget, over) :-
    TotalBudget > MaxBudget,
    format(atom(ErrorMessage), 'Total budget ~w EGP exceeds maximum budget ~w EGP (MinBudget: ~w EGP)', [TotalBudget, MaxBudget, MinBudget]),
    throw(error(budget_out_of_range, context(ErrorMessage))).
check_budget(TotalBudget, MaxBudget, MinBudget, under) :-
    TotalBudget < MinBudget * 0.8, % Definitely under the desired minimum
    format('Warning: Total budget ~w EGP is significantly below minimum desired budget ~w EGP (MaxBudget: ~w EGP).~n', [TotalBudget, MinBudget, MaxBudget]),
    !. % Succeed, but the status is 'under' for advice
```

Figure 23: check_budget/4

13.print_travel_plan/14

- **Structure:** `print_travel_plan(UserID, City, DurationDays, GroupSize, HotelName, Rating, TransportMode, TransportCost, DailyActivities, TotalBudget, PricePerNight, ActivityCosts, DailyFoodCost, DailyActivityDetails, BudgetStatus, AllActivitiesRaw, MinBudget, MaxBudget)`
- **Description:** Outputs a **formatted** *travel* plan with all details and **budget** advice.
- **Arguments:**
 - UserID: User's identifier.
 - City: **Destination** city.
 - DurationDays: *Trip* duration.
 - GroupSize: Number of travelers.
 - HotelName: Selected hotel.
 - Rating: Hotel rating.
 - TransportMode: Selected *transport*.
 - TransportCost: *Transport* cost.
 - DailyActivities: List of daily activity names.
 - TotalBudget: **Total** cost.
 - PricePerNight: Hotel cost per night.
 - ActivityCosts: Daily activity costs.
 - DailyFoodCost: Daily food cost.
 - DailyActivityDetails: Daily activity details.
 - BudgetStatus: **Budget** status.
 - AllActivitiesRaw: All **available** activities.
 - MinBudget, MaxBudget: **Budget** range.
- **Functionality:**
 - Displays *trip* overview (**destination**, duration, group size).
 - Lists **recommendations** (hotel, *transport*, activities) with alternatives.
 - Shows **associated** *attractions* using `activity_attraction_link/2`.
 - Provides a cost breakdown (hotel, activities, *transport*, food).
 - Prints a daily *itinerary* with hotel, *transport*, and activities.
 - Offers **budget** advice based on BudgetStatus.
- **Example:** Outputs a 3-day plan for Cairo with hotel, Metro, and activities like 'Pyramid Tour'.
- **Purpose:** Presents a **clear, user-friendly** *travel* plan with **comprehensive** details.


```
print_travel_plan(UserID, City, DurationDays, GroupSize, HotelName, Rating, TransportMode, TransportCost,
DailyActivities, TotalBudget, PricePerNight, ActivityCosts, DailyFoodCost, DailyActivityDetails, BudgetStatus, AllActivitiesRaw,
udget, MaxBudget) :-
format('~n== TRAVEL PLAN FOR ~w ==~n', [UserID]),
format(' Destination: ~w~n', [City]),
format(' Duration: ~w days~n', [DurationDays]),
format(' Group Size: ~w~n~n', [GroupSize]),

format('== RECOMMENDATIONS ==~n'),
format(' Selected Hotel: ~w (Rating: ~1f)~n', [HotelName, Rating]),
print_other_hotels(City, HotelName, DurationDays, TotalBudget), % Pass TotalBudget for context

format(' Selected Transport: ~w (Cost: ~w EGP)~n', [TransportMode, TransportCost]),
print_other_transports(City, TransportMode),

format(' Selected Activities:~n'),
( flatten(DailyActivities, FlatActivities) \= [] -> % Check if any activities were selected across all days
  print_selected_activities(DailyActivities, DailyActivityDetails, 1)
; format(' No activities selected (check budget/time constraints).~n')
),

print_other_activities(AllActivitiesRaw, DailyActivities, City),
print_associated_attractions(DailyActivities, City),

format('~n== COST ESTIMATE ==~n'),
TotalHotelCost is PricePerNight * DurationDays,
TotalFoodCost is DailyFoodCost * DurationDays,
( ActivityCosts = [] -> TotalActivityCost = 0 ; sum_list(ActivityCosts, TotalActivityCost) ),
format(' Hotel: ~w EGP~n', [TotalHotelCost]),
format(' Activities: ~w EGP~n', [TotalActivityCost]),
( flatten(DailyActivities, FlatActivitiesCheck), FlatActivitiesCheck \= [] ->
  print_activity_cost_details(1, DailyActivities, DailyActivityDetails, ActivityCosts)
; format(' No activities planned.~n')
),

format(' Transport: ~w EGP~n', [TransportCost]),
format(' Selected: ~w (~w EGP/trip)~n', [TransportMode, TransportCost]),
format(' Food: ~w EGP~n', [TotalFoodCost]),
format(' Daily: ~w EGP (~w days)~n', [DailyFoodCost, DurationDays]),
format(' TOTAL: ~w EGP~n~n', [TotalBudget]),

format('== DAILY ITINERARY ==~n'),
( flatten(DailyActivities, FlatItinCheck), FlatItinCheck \= [] ->
```

Figure 24: print_travel_plan/14

14.submit_hotel_review/4

- **Structure:** submit_hotel_review(UserID, HotelName, Rating, Comment)
- **Description:** Allows users to submit hotel reviews, dynamically updating the *knowledge base*.
- **Arguments:**
 - UserID: User's identifier.
 - HotelName: Hotel name.
 - Rating: Review rating (0–5).
 - Comment: Review comment.
- **Functionality:**
 - Validates that HotelName exists in hotel/5.
 - Ensures Rating is a number between 0 and 5.
 - Checks that Comment is a non-empty atom or string.
 - Asserts the review to hotel_review/3 using assertz/1.
 - Outputs confirmation of submission.
- **Example:** submit_hotel_review('Omar', 'Hilton Cairo Nile', 5.0, 'Perfect stay!')
- **Purpose:** Enhances hotel selection with **user** feedback.

```
% submit_hotel_review(UserID, HotelName, Rating, Comment)
% Submits a hotel review.
submit_hotel_review(UserID, HotelName, Rating, Comment) :-
    ( hotel(_, HotelName, _, _, _) ->
        true
    ; format(atom(ErrMsg), 'Hotel ~w does not exist', [HotelName]),
      throw(error(invalid_hotel, context(ErrMsg)))
    ),
    ( number(Rating), Rating >= 0, Rating <= 5 ->
        true
    ; throw(error(invalid_rating, context('Rating must be a number between 0 and 5')))
    ),
    ( (atom(Comment); string(Comment)), Comment \= '', Comment \= "" ->
        true
    ; throw(error(invalid_comment, context('Comment cannot be empty')))
    ),
    assertz(hotel_review(HotelName, Rating, Comment)),
    format('Review submitted by ~w for ~w: ~lf stars, "~w"~n', [UserID, HotelName, Rating, Comment]).
```

Figure 25: submit_hotel_review/4

15.update_hotel/5

- **Structure:** update_hotel(UserID, City, OldHotelName, NewHotelName, NewPrice)
- **Description:** Updates a hotel's name and price, retaining its rating and demand.
- **Arguments:**
 - UserID: User's identifier (e.g., admin).
 - City: Hotel's city.
 - OldHotelName: **Current** hotel name.
 - NewHotelName: New hotel name.
 - NewPrice: New price per night.
- **Functionality:**
 - Verifies OldHotelName exists in hotel/5 for the city.
 - Ensures NewPrice is positive.
 - Checks that NewHotelName is **unique** or matches OldHotelName.
 - Retracts the old hotel fact and asserts the **updated** one.
 - Outputs confirmation of the update.
- **Example:** update_hotel('Admin', 'Cairo', 'Economy Hotel Cairo', 'Super Economy Cairo', 550)
- **Purpose:** Maintains an **up-to-date knowledge base** for hotels.

```
% update_hotel(UserID, City, OldHotelName, NewHotelName, NewPrice)
% Replaces a hotel's name and price. Retains rating and demand.
update_hotel(UserID, City, OldHotelName, NewHotelName, NewPrice) :-
    ( hotel(City, OldHotelName, OldPrice, OldRating, OldDemand) ->
        true
    ; format(atom(ErrMsg), 'Hotel ~w does not exist in ~w', [OldHotelName, City]),
      throw(error(invalid_hotel, context(ErrMsg)))
    ),
    ( number(NewPrice), NewPrice > 0 ->
        true
    ; throw(error(invalid_price, context('New price must be a positive number')))
    ),
    ( OldHotelName == NewHotelName ; \+ hotel(_, NewHotelName, _, _, _) -> % Allow updating price for the same name
        true
    ; format(atom(ErrMsg), 'Hotel name ~w is already in use', [NewHotelName]),
      throw(error(duplicate_hotel, context(ErrMsg)))
    ),
    retract(hotel(City, OldHotelName, OldPrice, OldRating, OldDemand)),
    assertz(hotel(City, NewHotelName, NewPrice, OldRating, OldDemand)), % Use old rating and demand
    format('Hotel ~w in ~w updated to ~w (~w EGP/night) by ~w~n', [OldHotelName, City, NewHotelName, NewPrice, UserID])
```

Figure 26: update_hotel/5

The Output in SWI-Prolog:

```

travel_planner_4.pl
File Edit Browse Compile Prolog Pce Help
travel_planner_4.pl
um'). Military Museum Visit', 'PortSaid Military Muse
activity_attraction_link('Corniche Evening Walk', 'PortSaid Corniche').

% --- Rules ---
% =====
% Interactive Planner Start Predicate
% =====
start_planner :-
    write('--- Welcome to the Travel Planner ---'), nl,
    prompt_read_string('Enter your name (UserID): ', UserID),
    prompt_read_city('Enter destination city (Cairo, Aswan, PortSaid): ',
City),
    prompt_read_integer('Enter trip duration in days: ', DurationDays),
    prompt_read_month('Enter travel month (e.g., January, February): ', M
onth),
    prompt_read_hours('Enter max activity hours per day (4, 6, 8, 10): ',
MaxHoursPerDay),
    prompt_read_integer('Enter group size: ', GroupSize),
    prompt_read_number('Enter minimum budget (EGP): ', MinBudget),
    prompt_read_max_budget('Enter maximum budget (EGP): ', MinBudget, Max
Budget),
    nl, write('--- Generating Plan... ---'), nl, nl,
    % Call the main planner, catch errors locally
    catch(
        plan_trip(UserID, City, DurationDays, Month, MaxHoursPerDay, Grou
pSize, MinBudget, MaxBudget),
        Error,
        ( write('Error during planning: '), writeln(Error),
          write('Failed to generate a complete plan. Please check input
s or adjust constraints.'), nl
        )
    ),
    nl, write('--- Planner Finished ---'), nl.

% Helper predicates for reading validated input
prompt_read_string(Prompt, Value) :-
    write(Prompt), flush_output,
    read_line_to_string(user_input, Value),
    ( Value == "" -> true ; write('Input cannot be empty. Please try agai
n.'), nl, prompt_read_string(Prompt, Value) ).
comment(line)
Line: 198

```

```

SWI-Prolog (AMD64, Multi-threaded, version 9.3.20)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 9.3.20)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?-
% library(apply) compiled into apply 0.00 sec. 75 clauses
% h:/Prolog/Projects/travel_planner_4.pl compiled into travel_planner_4 0.02 sec. 218 claus
es
?- start_planner
--- Welcome to the Travel Planner ---
Enter your name (UserID): Nardin
Enter destination city (Cairo, Aswan, PortSaid): Cairo
Enter trip duration in days: 3
Enter travel month (e.g., January, February): April
Enter max activity hours per day (4, 6, 8, 10): 8
Enter group size: 5
Enter minimum budget (EGP): 1000
Enter maximum budget (EGP): 10000
--- Generating Plan... ---

*** TRAVEL PLAN FOR Nardin ***
Destination: Cairo
Duration: 3 days
Group Size: 5

*** RECOMMENDATIONS ***
Selected Hotel: Budget Hotel Cairo (Rating: 4.0)
Other Hotels:
Economy Hotel Cairo: 600 EGP/night, Rating: 3.8
Selected Transport: Bus (Cost: 8 EGP)
Other Transports:
Metro: 10 EGP/trip
Train: 40 EGP/trip
Taxi: 50 EGP/trip
Selected Activities:
Day 1:
Khan el-Khalili Bazaar: 100 EGP, 2.0 hours
Street Food Tour: 100 EGP, 2.0 hours
Coptic Cairo Walking Tour: 150 EGP, 2.5 hours
Camel Ride at Giza: 200 EGP, 1.0 hours
Day 2:
Egyptian Museum Visit: 200 EGP, 3.0 hours
Sound and Light Show at Pyramids: 250 EGP, 1.5 hours
Nile River Cruise: 300 EGP, 2.0 hours
Day 3:
Citadel and Mohamed Ali Mosque Tour: 350 EGP, 3.0 hours
Pyramid Tour: 500 EGP, 4.0 hours
Other Activities:
Desert Quad Biking: 600 EGP, 3.0 hours (Attraction: Giza Pyramids)
Associated Attractions:
Khan el-Khalili Bazaar (tourist_spot): Open 9:00 - 22:00
Coptic Cairo Walking Tour (cultural_site): Open 9:00 - 17:00
Camel Ride at Giza (landmark): Open 8:00 - 17:00
Egyptian Museum Visit (cultural_site): Open 9:00 - 16:00
Citadel and Mohamed Ali Mosque Tour (cultural_site): Open 8:00 - 16:00
Pyramid Tour (landmark): Open 8:00 - 17:00

```

```

*** COST ESTIMATE ***
Hotel: 2400 EGP
Activities: 2150 EGP
Day 1: 550 EGP
Khan el-Khalili Bazaar: 100 EGP, 2.0 hours
Street Food Tour: 100 EGP, 2.0 hours
Coptic Cairo Walking Tour: 150 EGP, 2.5 hours
Camel Ride at Giza: 200 EGP, 1.0 hours
Day 2: 750 EGP
Egyptian Museum Visit: 200 EGP, 3.0 hours
Sound and Light Show at Pyramids: 250 EGP, 1.5 hours
Nile River Cruise: 300 EGP, 2.0 hours
Day 3: 850 EGP
Citadel and Mohamed Ali Mosque Tour: 350 EGP, 3.0 hours
Pyramid Tour: 500 EGP, 4.0 hours
Transport: 8 EGP
Selected Bus (8 EGP/trip)
Food: 1200 EGP
Daily: 400 EGP (3 days)
TOTAL: 5758 EGP

*** DAILY ITINERARY ***
DAY 1:
Hotel: Budget Hotel Cairo
Transport: Bus
Activities:
Khan el-Khalili Bazaar: 2.0 hours
Street Food Tour: 2.0 hours
Coptic Cairo Walking Tour: 2.5 hours
Camel Ride at Giza: 1.0 hours
DAY 2:
Hotel: Budget Hotel Cairo
Transport: Bus
Activities:
Egyptian Museum Visit: 3.0 hours
Sound and Light Show at Pyramids: 1.5 hours
Nile River Cruise: 2.0 hours
DAY 3:
Hotel: Budget Hotel Cairo
Transport: Bus
Activities:
Citadel and Mohamed Ali Mosque Tour: 3.0 hours
Pyramid Tour: 4.0 hours
Plan saved successfully for Nardin.

*** BUDGET ADVICE ***
Your plan costs 5758 EGP, leaving 4242 EGP within your max budget for flexibility or upgr
ades.
Consider upgrading the hotel, adding premium activities, or enjoying finer dining.

--- Planner Finished ---
true.
?-

```

The Output with using GUI:

Egyptian Travel Planner
Your Ultimate Egypt Vacation Guide

Plan Your Trip | Hotel Management

Traveler Name: Destination:

Duration (days): Travel Month:

Daily Activity Hours: Group Size:

Min Budget (EGP): Max Budget (EGP):

Generate Plan (Ctrl+G)

Trip Plan Results Save Plan Print

```

=== TRAVEL PLAN FOR Nardin ===
Destination: Cairo (Month: April)
Duration: 3 days
Group Size: 5

=== SELECTIONS ===
Hotel: Budget Hotel Cairo (Rating: 4.0, 800 EGP/night)
Transport: Bus (8 EGP)
Est. Daily Food Cost: 400 EGP

=== OTHER HOTEL OPTIONS ===
- Economy Hotel Cairo (Rating: 3.8, 600 EGP/night)
- Sofitel Cairo Downtown Nile (Rating: 4.6, 1100 EGP/night)

=== COST ESTIMATE ===
Hotel: 2400 EGP
Activities: 2150 EGP
Transport: 8 EGP
Food: 1200 EGP
    
```

Egyptian Travel Planner
Your Ultimate Egypt Vacation Guide

Plan Your Trip | Hotel Management

Traveler Name: Destination:

Duration (days): Travel Month:

Daily Activity Hours: Group Size:

Min Budget (EGP): Max Budget (EGP):

Generate Plan (Ctrl+G)

Trip Plan Results Save Plan Print

```

Transport: 8 EGP
Food: 1200 EGP
-----
TOTAL: 5758 EGP
Budget Status: Fits within budget guidelines.

=== DAILY ITINERARY & DETAILS ===
--- DAY 1 (Cost: 550 EGP) ---
- Khan el-Khalili Bazaar (2 hrs, 100 EGP)
- Street Food Tour (2 hrs, 100 EGP)
- Coptic Cairo Walking Tour (2.5 hrs, 150 EGP)
- Camel Ride at Giza (1 hrs, 200 EGP)

--- DAY 2 (Cost: 750 EGP) ---
- Egyptian Museum Visit (3 hrs, 200 EGP)
- Sound and Light Show at Pyramids (1.5 hrs, 250 EGP)
- Nile River Cruise (2 hrs, 300 EGP)

--- DAY 3 (Cost: 850 EGP) ---
- Citadel and Mohamed Ali Mosque Tour (3 hrs, 350 EGP)
    
```


1. Diagrams

3.1 Use Case Diagram

The Use Case Diagram for the **Logic-based Travel Planner** (illustrated in **Figure 1**) provides a visual representation of the functional requirements and interactions between the system's actors and its core processes. This diagram outlines the primary functionalities offered to the **User** and the supporting roles of the **System** and **Knowledge Base** actors, ensuring comprehensive travel planning experience. Below is a detailed explanation of the diagram's components and their relationships.

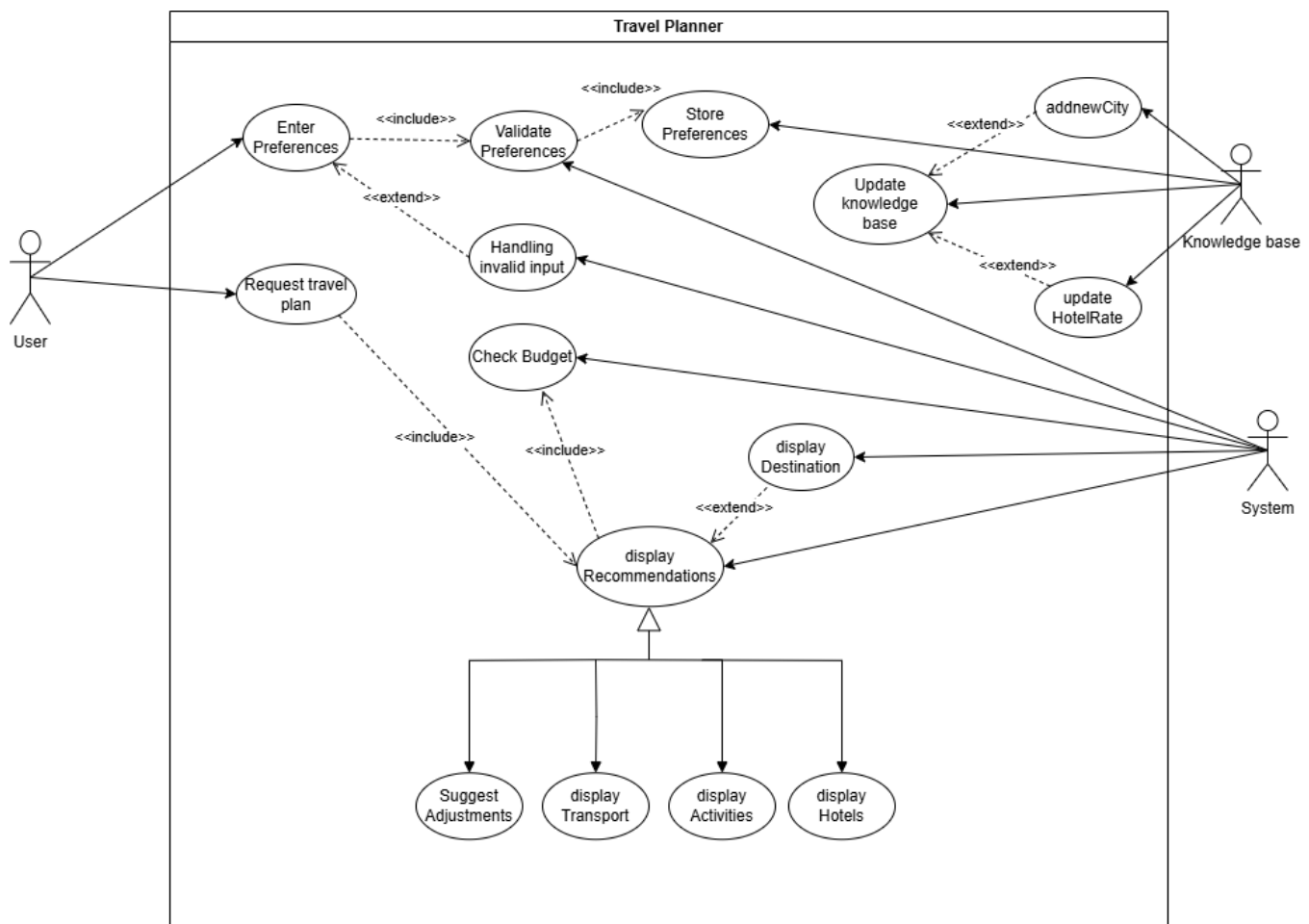


Figure 27: Use case diagram

3.1.1 Actors

- **User:** Represents the end-user who interacts with the system to plan their travel. The **User** initiates all primary actions, such as entering preferences and requesting travel plans.
- **System:** Acts as the operational backbone, executing processes like validation, recommendation generation, and knowledge base updates in response to user inputs.
- **Knowledge Base:** Serves as a dynamic repository that stores and updates travel-related data (e.g., cities, hotels, attractions), interacting with the **System** to provide and refine information.

3.1.2 Primary Use Cases

1. Enter Preferences:

- The **User** begins by entering their travel preferences (e.g., trip type, budget, duration), which is a foundational step in the planning process. This use case includes:
 - **Validate Preferences:** The **System** validates the entered preferences (e.g., checking budget feasibility) to ensure they are actionable.
 - **Store Preferences:** Validated preferences are stored in the **Knowledge Base** for later use in generating travel plans.
- **Relationship:** The <<include>> relationship indicates that validation and storage are mandatory subprocesses of entering preferences.

2. Request Travel Plan:

- The **User** requests a tailored travel plan based on their stored preferences. This triggers a series of dependent actions:
 - **display Destination:** The **System** provides information about potential destinations.
 - **Show Recommendations:** The core outcome, displaying a comprehensive plan including hotels, transport, and activities.
- **Relationship:** The <<include>> relationship ensures that destination display and recommendations are integral to the request process.

3. Show Recommendations:

- This central use case generates and presents the travel plan, incorporating:
 - **Check Budget:** The **System** verifies that the plan aligns with the user's budget.
 - **display Transport:** Suggests transportation options.
 - **display Activities:** Lists available activities (e.g., shopping, museums).
 - **display Hotels:** Recommends suitable accommodations.
- **Extension: Suggest Adjustments** (<<extend>>) is triggered if the budget is invalid, allowing the **User** to modify their preferences.
- **Relationship:** The <<include>> relationships highlight the multi-faceted nature of recommendations, while the extension provides flexibility.

4. Update Knowledge Base:

- The **System** updates the **Knowledge Base** to reflect new data or changes, including:
 - **addnewCity** (<<extend>>): Adds a new city (e.g., Luxor) to expand the system's scope.
 - **updateHotelRate** (<<extend>>): Adjusts hotel ratings or prices (e.g., updating Four Seasons to 2200 EGP).
- **Relationship:** The <<extend>> relationships indicate optional enhancements to the knowledge base.

3.1.3 Extended Use Cases

- **Handling Invalid Input** (<<extend>> from **Enter Preferences**):

- If the **User** provides invalid preferences (e.g., insufficient budget), the **System** handles the input error, prompting the **User** to re-enter data.
 - **Suggest Adjustments** (<<extend>> from **Show Recommendations**):
 - Activated when the **Check Budget** process identifies an invalid budget, offering the **User** suggestions to adjust their plan.
-

3.2 Sequence Diagram

The Sequence Diagram for the **Logic-based Travel Planner** (illustrated in **Figure 2**) depicts the dynamic interaction between the **User**, **System**, **Budget System**, and **Knowledge Base** actors over time. It illustrates the step-by-step process of entering preferences, generating a travel plan, checking budget constraints, and updating the knowledge base, with activation bars highlighting active processing periods. Below is a detailed explanation of the diagram's flow and its significance.

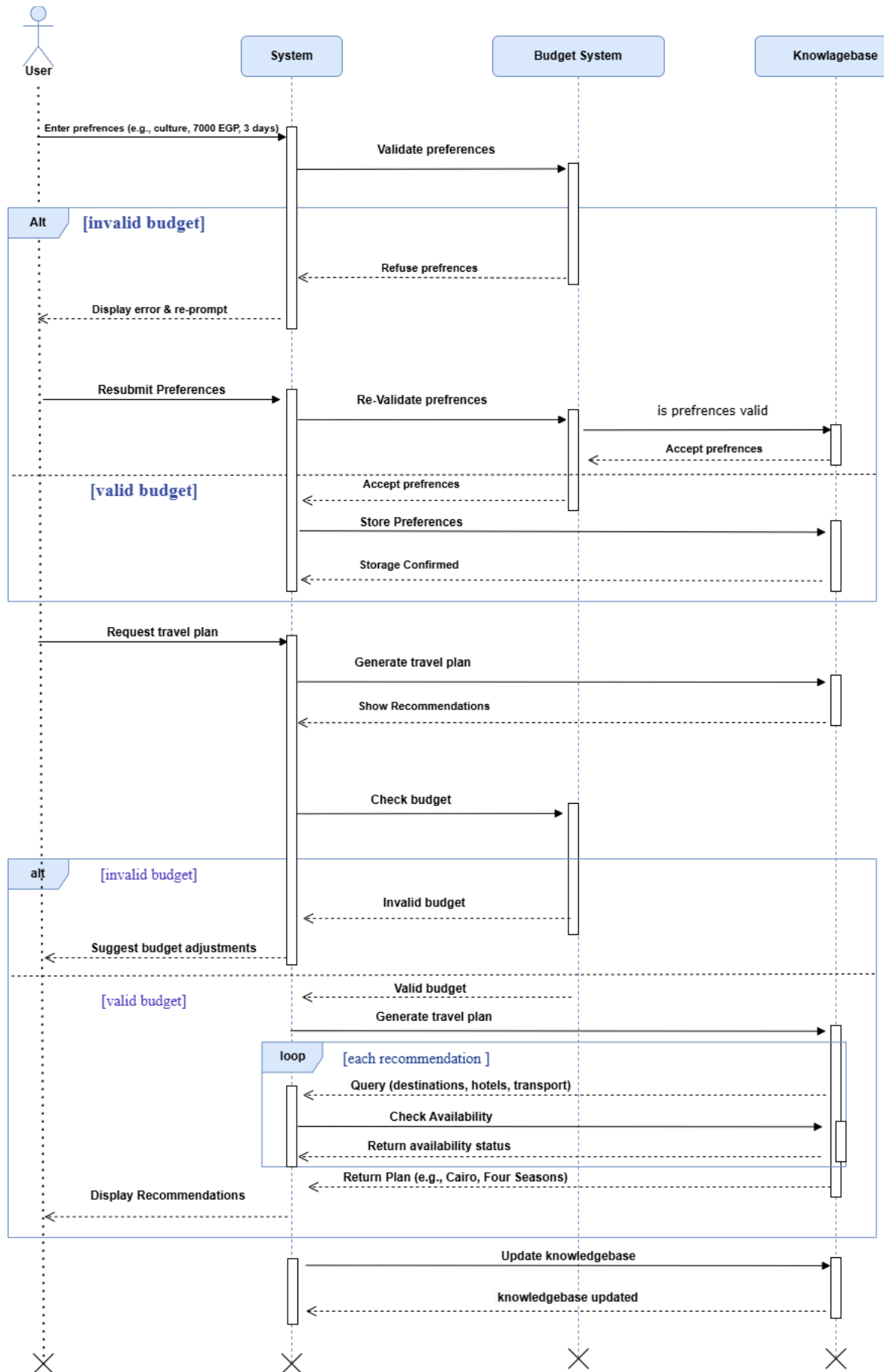


Figure 28: Sequence diagram

3.2.1 Actors and Participants

- **User:** The initiator of the process, sending requests and receiving responses.
- **System:** The central coordinator, managing interactions with the **Budget System** and **Knowledge Base**.
- **Budget System:** A specialized component responsible for validating and checking budget constraints.
- **Knowledge Base:** A data repository that stores and retrieves travel-related information (e.g., cities, hotels, preferences).

3.2.2 Sequence Flow

1. Entering Preferences:

- The **User** sends a message "**Enter preferences (e.g., culture, 7000 EGP, 3 days)**" to the **System**.
- The **System** activates and forwards the request to the **Budget System** with "**Validate preferences**".
- **Alternative Flow ([Invalid budget]):**
 - If the budget is invalid, the **Budget System** returns "**Refuse preferences**" to the **System**.
 - The **System** responds to the **User** with "**Display error & re-prompt**", prompting resubmission.
 - The **User** resubmits preferences ("**Resubmit Preferences**"), and the **System** re-engages the **Budget System** with "**Re-validate preferences**".
 - If still invalid, the process exits with "**Log error & exit**"; otherwise, it proceeds.
- **Main Flow ([Valid budget]):**
 - If valid, the **Budget System** sends "**Accept preferences**" to the **System**.
 - The **System** instructs the **Knowledge Base** to "**Store preferences**", receiving "**Storage confirmed**" in return.

2. Requesting Travel Plan:

- The **User** sends "**Request travel plan**" to the **System**.
- The **System** retrieves data from the **Knowledge Base** with "**Retrieve preferences from KnowledgeBase**", receiving the stored preferences.
- The **System** then commands the **Knowledge Base** to "**Generate travel plan**", which returns "**Show Recommendations**".

3. Budget Checking and Plan Generation:

- The **System** sends "**Check budget**" to the **Budget System**.
- **Alternative Flow ([Invalid budget]):**
 - If invalid, the **Budget System** returns "**Invalid budget**", and the **System** suggests "**Suggest budget adjustments**" to the **User**.
- **Main Flow ([Valid budget]):**
 - If valid, the **Budget System** returns "**Valid budget**", and the **System** initiates a **loop** for each recommendation:

- The **Knowledge Base** queries "**Query (destinations, hotels, transport)**".
- The **System** checks "**Check availability**" with the **Knowledge Base**, which responds with "**Return availability status**".
- The loop concludes with the **Knowledge Base** returning "**Return plan (e.g., Cairo, Four Seasons)**".
- The **System** then sends "**Display recommendations**" to the **User**.

4. Updating Knowledge Base:

- The **System** sends "**Update knowledgebase**" to the **Knowledge Base**.
- **Alternative Flow:**
 - If a **new city** is added ("**Add new city (e.g., Luxor)**") or a **hotel rate** is updated ("**Update hotel rate (e.g., Four Seasons to 2200)**"), the **Knowledge Base** processes accordingly.
- The **Knowledge Base** confirms with "**Knowledgebase updated**", and the **System** deactivates.

3.2.3 Control Structures

- **Alt ([Invalid budget]):** Handles budget validation with two branches—re-prompting the **User** or exiting if unresolved.
- **Loop ([Each recommendation]):** Iterates over destination, hotel, and transport queries to compile the travel plan.
- Activation bars indicate the duration of processing for each actor, with solid lines for requests and dashed lines for responses.

3.3 Activity Diagram

The Activity Diagram for the **Logic-based Travel Planner** (illustrated in **Figure 3**) models the procedural workflow of the system, from entering user preferences to generating and displaying a travel plan, including budget validation and knowledge base updates. It captures the sequence of activities, decision points, loops, and conditional paths, providing a comprehensive view of the system's operational logic. Below is a detailed explanation of the diagram's flow and its significance.

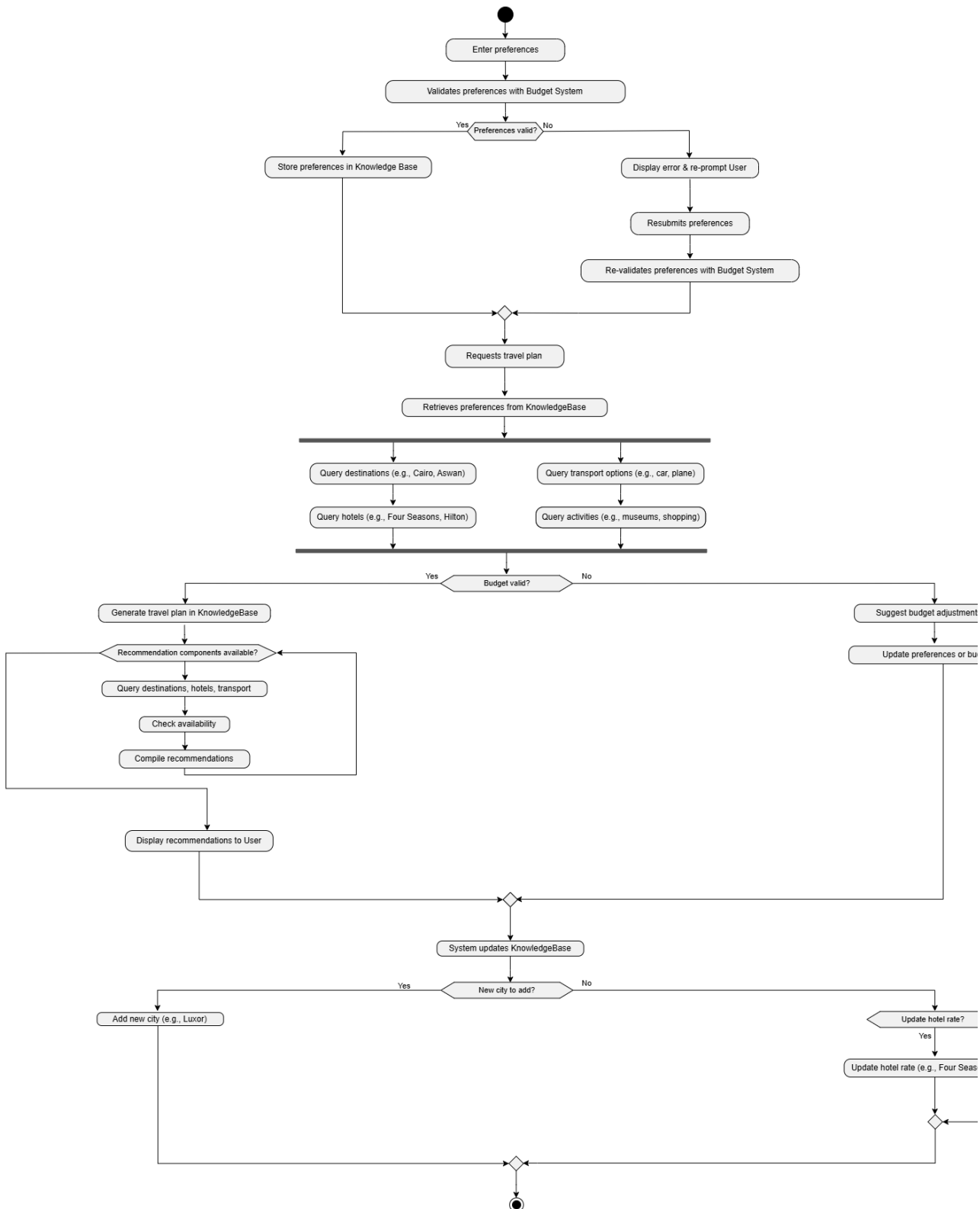


Figure 29: Activity diagram

3.3.1 Workflow Overview

The diagram kicks off with a **start node** and wraps up with a **stop node**, illustrating the full journey of travel planning. It brings together user interactions, system processes, and updates to the knowledge base, featuring decision points to tackle validation and budget limits.

3.3.2 Activity Flow

1. Enter Preferences:

- The process starts with the activity **"Enter preferences in System"**, where the **User** inputs preferences (e.g., trip type, budget, duration).
- This leads to **"Validate preferences with Budget System"**, a decision point:
 - **Yes (Preferences Valid):** Proceeds to **"Store preferences in KnowledgeBase"**, saving the data for later use.
 - **No (Preferences Invalid):** Triggers **"Display error with Budget System"**, followed by **"Re-prompt User"** and **"Resubmit preferences"**.
 - A nested decision **"Re-validate preferences with Budget System"** checks the resubmitted data:
 - **Yes (Valid):** Continues to **"Store preferences in KnowledgeBase"**.
 - **No (Still Invalid):** Exits with **"Log error & exit"**, ending the process.

2. Request Travel Plan:

- After storing preferences, the **User** initiates **"Request travel plan"**.
- The **System** performs **"Retrieve preferences from KnowledgeBase"** to access the stored data.
- This is followed by **"Generate travel plan in KnowledgeBase"**, initiating the core planning process.

3. Generate Travel Plan:

- The **Knowledge Base** executes a series of queries:
 - **"Query destinations (e.g., Cairo, Aswan)"**, **"Query hotels (e.g., Four Seasons, Hilton)"**, **"Query transport"**, and **"Query activities (e.g., museums, shopping)"**.
- These queries feed into **"Generate travel plan in KnowledgeBase"**, which consolidates the data.

4. Budget Validation:

- A decision point **"Budget valid?"** evaluates the generated plan:
 - **Yes (Valid Budget):** Proceeds to a **loop** for recommendation compilation:
 - **"Query (destinations, hotels, transport)"**: Collects data for each component.
 - **"Check availability"**: Verifies the feasibility of the selections.
 - The loop iterates until all components are processed, then moves to **"Compile recommendations"** and **"Display recommendations to User"**.
 - **No (Invalid Budget):** Triggers **"Suggest budget adjustments to User"**, followed by **"Update preferences or budget"**, looping back to **"Generate travel plan in KnowledgeBase"** to refine the plan.

5. Update Knowledge Base:

- The **System** initiates **"System updates KnowledgeBase"**, followed by a decision:
 - **"New city added?"**:

- **Yes:** Executes "**Add new city (e.g., Luxor)**".
- **No:** Proceeds to "**Update hotel rate?**":
 - **Yes:** Performs "**Update hotel rate (e.g., Four Seasons to 2200)**".
 - **No:** Skips further updates.
- The process concludes with "**KnowledgeBase updated**", ensuring the system remains current.

6. Finalization:

- The workflow ends with the final node, indicating completion of the travel planning process.

3.3.3 Control Structures

- **Decision Points:**
 - "**Validate preferences with Budget System?**" and "**Re-validate preferences with Budget System?**" ensure robust input validation.
 - "**Budget valid?**" handles budget constraints, offering a feedback loop for adjustments.
 - "**New city added?**" and "**Update hotel rate?**" manage knowledge base updates dynamically.
- **Loop:** The "**Query (destinations, hotels, transport)**" loop ensures all components of the travel plan are processed iteratively, enhancing *efficiency*.
- **Swimlanes:** The diagram implicitly uses swimlanes to separate responsibilities between the **System**, **User**, and **Knowledge Base**, though not explicitly labeled.

4. Conclusion

The **Logic-based Travel Planner** is a *big step forward* in using **logic programming** to tackle the many challenges of travel planning. This mid-term submission delivers a *strong, rule-based system* that combines **budget estimation**, **hotel recommendations** (through the `hotel/4` predicate), **destination details**, **transportation support**, and **activity discovery** within a **Prolog framework**. The system's main features—shown in the **Use Case Diagram**, **Sequence Diagram**, and **Activity Diagram**—emphasize a *user-friendly design* that guarantees *customized, budget-friendly travel plans*. By confirming user preferences, creating personalized suggestions, and updating the **Knowledge Base** in real time, the system makes the travel planning process smoother and turns a traditionally *time-consuming* task into an *efficient, knowledgeable experience*.

The way we've set things up uses **Prolog's smart reasoning abilities** along with a modular *knowledge base*, which gives us a great foundation for scaling things up and ensuring reliability. The introduction of rules like `estimated_budget/3` and `personalized_recommendation/4` shows how the system can handle complex limits, while the **CLI interface** gives a practical platform for testing and validation. The diagrams show how flexible the system really is. They help with everything from handling incorrect inputs to suggesting budget adjustments, making sure that users have a *hassle-free experience*.

Looking ahead, the **Logic-based Travel Planner** is ready for more upgrades in the final phase. Future versions will enhance the **Knowledge Base** with *real-time data* (such as seasonal prices, user feedback), improve rules for *multi-day itineraries*, and add a **graphical user interface (GUI)** to boost *accessibility*. New features, like allowing *multiple users* and better activity scheduling, are going to make the system even more useful. This project not only meets the goals of the *Logic Programming course* but also demonstrates the *real-world application* of **Prolog** in solving practical problems, paving the way for future advancements in *intelligent travel planning systems*.

5. References

1. Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
2. Bratko, I. (2012). *Prolog Programming for Artificial Intelligence* (4th ed.). Addison-Wesley.
3. Object Management Group (OMG). (2023). *Unified Modeling Language (UML) Specification, Version 2.5.1*. Retrieved from <https://www.omg.org/spec/UML/2.5.1>
4. SWI-Prolog. (2023). *SWI-Prolog Documentation*. Retrieved from <https://www.swi-prolog.org/>
5. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.
6. [Prof. Dr. Hesham El-Deeb's book](#)

6. Our Project in GitHub

[HosamDyab/logic-based-travel-planner-prolog](https://github.com/HosamDyab/logic-based-travel-planner-prolog)